

# Analisi di Co-occorrenza di Terremoti con Scala e Apache Spark

Progetto — Scalable and Cloud Programming, A.A. 2025-26

Alberto Zini — 0001007558

Repository: <https://github.com/Sh0wBlack32/SCP-earthquake-cooccurrence>

## 1. Descrizione del problema

L'obiettivo del progetto è realizzare, in Scala e Apache Spark, un'analisi di co-occorrenza di eventi sismici in località diverse. Data una serie storica di terremoti, ciascuno caratterizzato da latitudine, longitudine e istante temporale, si vuole individuare la coppia di località che co-occorrono più spesso, ovvero la coppia di aree in cui si registrano eventi sismici nella stessa finestra temporale (un giorno) nel maggior numero di giorni distinti. Il risultato richiesto è la coppia vincente e l'elenco, in ordine crescente, delle date in cui i due eventi co-occorrono.

Poiché la posizione esatta di un evento non è rilevante quanto l'area in cui esso si verifica, latitudine e longitudine vengono arrotondate alla prima cifra decimale (arrotondamento al valore più vicino). Questo introduce duplicati — più eventi nello stesso giorno che ricadono nella stessa “cella” geografica arrotondata — che devono essere deduplicati prima di generare le coppie, altrimenti si rischia di individuare come vincente una coppia di eventi nella stessa cella geografica.

### 1.1 Dataset

Il dataset è composto da record (latitude, longitude, date) in formato CSV con intestazione. Sono disponibili due versioni: una versione ridotta, usata per i test in locale e sulla configurazione solo-master, e una versione completa (1.47M record), usata per l'esecuzione e la valutazione delle prestazioni su Google Cloud DataProc.

### 1.2 Risultato

Sul dataset completo, entrambi gli approcci individuano la stessa coppia vincente — **((38.8, -122.8), (38.8, -122.7))** — con **10032 date di co-occorrenza**, dal 1990-01-05 al 2023-07-29 (33.6 anni). Le coordinate ricadono nell'area di The Geysers, in California del Nord, il più grande campo geotermico al mondo, dove l'estrazione geotermica induce una sismicità indotta pressoché quotidiana da decenni: la coppia co-occorre nell'81.8% dei giorni dell'intero intervallo, un risultato coerente col dominio applicativo e non un artefatto dei dati o dell'algoritmo.

## 2. Approccio implementativo

L'elaborazione segue lo schema map-reduce ed è interamente distribuita tramite RDD di Spark. La pipeline comune ai due approcci è la seguente:

- lettura del CSV e arrotondamento di lat/lon alla prima cifra decimale;
- troncamento del timestamp alla data (YYYY-MM-DD);
- deduplica delle coppie (località, data) generate dall'arrotondamento;
- generazione, per ogni giorno, di tutte le coppie canoniche di località (loc\_i, loc\_j) con loc\_i < loc\_j;
- deduplica delle coppie (coppia, data);
- conteggio delle co-occorrenze per coppia e individuazione del massimo;
- raccolta e ordinamento delle date per la coppia vincente.

### 2.1 Approccio 1 — groupByKey

Le località vengono raggruppate per data con **groupByKey**; per ogni giorno, le coppie candidate vengono generate localmente (in memoria, sul singolo executor) a partire dalla lista di località distinte osservate quel giorno. È l'approccio più efficiente quando il numero di eventi per giorno è contenuto, poiché evita uno shuffle aggiuntivo per generare le coppie.

## 2.2 Approccio 2 — Self-Join

In alternativa, le coppie vengono generate tramite un self-join distribuito dell'RDD chiave-valore (data, località) con se stesso, filtrando poi le coppie con  $loc1 < loc2$  per eliminare simmetrie e auto-coppie. Questo approccio distribuisce il lavoro di generazione delle coppie su tutto il cluster invece di concentrarlo sul singolo executor che gestisce un dato giorno, a costo di uno shuffle più oneroso; può risultare preferibile quando i giorni con molti eventi (skew) rendono costoso il groupByKey.

## 3. Setup sperimentale

Le prove sono state eseguite su Google Cloud DataProc (regione europe-west1), con cluster di 2, 3 e 4 worker, boot disk da 240 GB, memoria executor/driver aumentata esplicitamente rispetto ai default di DataProc (necessario per contenere lo shuffle spill sulla chiave skewata — vedi Sezione 4). Il cluster a 2 worker usa macchine **n2-standard-4** (4 vCPU/16 GB); per i cluster a 3 e 4 worker è stato necessario passare a **n2-standard-2** (2 vCPU/8 GB) a causa di un vincolo di quota GCP scoperto durante gli esperimenti stessi, descritto in dettaglio in Sezione 4. Il dataset completo e il JAR dell'applicazione sono stati caricati su un bucket Google Cloud Storage e referenziati tramite URI gs:// nei job Spark.

Le varianti con repartition(16)/repartition(32) sul cluster a 4 worker, previste nel piano sperimentale, non sono state eseguite per limiti di tempo e di quota (vedi nota sotto la tabella dei tempi): la quota globale del progetto education si è rivelata sufficiente solo per un numero limitato di run cloud completi, e si è scelto di privilegiare la copertura dei tre punti di scalabilità sul numero di worker rispetto alle varianti di partizionamento.

### Versioni e librerie

| Componente   | Versione                             |
|--------------|--------------------------------------|
| Scala        | 2.12.17                              |
| Apache Spark | 3.3.2                                |
| sbt          | 1.9.7                                |
| Piattaforma  | Google Cloud DataProc (europe-west1) |

## 4. Analisi di scalabilità e prestazioni

Durante gli esperimenti è emerso un vincolo non documentato in anticipo: il progetto GCP education-tier impone una quota **globale** (non solo per regione) di 12 vCPU per l'intero progetto (metrica `compute.googleapis.com/cpus_all_regions`, verificabile con `gcloud compute project-info describe`). Con n2-standard-4, un cluster a 2 worker (1 master + 2 worker  $\times$  4 vCPU) usa già l'intera quota disponibile: non è possibile creare cluster a 3 o 4 worker con lo stesso tipo di macchina senza un aumento di quota, che si è rivelato non ottenibile automaticamente su un progetto così recente (motivo restituito dall'API: `NOT_ENOUGH_USAGE_HISTORY`). Si è quindi proceduto usando n2-standard-2 (2 vCPU / 8 GB) per i cluster a 3 e 4 worker, che introduce una variabile di confondimento esplicita — i tre punti della curva non condividono lo stesso hardware per nodo — dichiarata qui per trasparenza metodologica.

| Worker (macchina) | Partizioni | Approccio  | Tempo (s)          |
|-------------------|------------|------------|--------------------|
| 2 (n2-std-4)      | default    | groupByKey | 4451.7 (74.2 min)  |
| 2 (n2-std-4)      | default    | self-join  | 4619.0 (77.0 min)  |
| 3 (n2-std-2)*     | default    | groupByKey | 9647.2 (160.8 min) |

| Worker (macchina) | Partizioni | Approccio  | Tempo (s)           |
|-------------------|------------|------------|---------------------|
| 4 (n2-std-2)*     | default    | groupByKey | 11830.1 (197.2 min) |

\* macchina più piccola (n2-standard-2) per il vincolo di quota descritto sopra, non per scelta progettuale.

## 4.1 Scalabilità (strong scaling)

Il risultato più significativo non è la velocità assoluta, ma la sua direzione: passando da 3 a 4 worker il tempo di esecuzione **aumenta** (da 160.8 a 197.2 minuti) invece di diminuire, nonostante il cluster a 4 worker abbia più nodi. Questo è coerente con la natura del dataset: la coppia vincente co-occorre nell'81.8% dei giorni in un intervallo di 33.6 anni (10032 su 12259 giorni), quindi la data-chiave a cui è associata gran parte del lavoro di generazione/deduplica delle coppie è fortemente skewata. Sia `groupByKey` sia `.join()` di Spark RDD ripartizionano per chiave: tutti i valori di una singola chiave finiscono in un numero di task che non cresce aggiungendo worker (a differenza delle `DataFrame Spark SQL`, dove l'Adaptive Query Execution gestisce lo skew-join automaticamente ridistribuendo la chiave calda su più task). Aggiungere nodi, in questo scenario, aumenta solo l'overhead di coordinamento e di shuffle di rete tra più macchine, senza alcun guadagno di parallelismo reale sulla parte di lavoro che conta di più — da qui il rallentamento anziché lo speedup atteso da un naive strong scaling. Il confronto è inoltre confondato dal cambio di macchina (n2-standard-4 → n2-standard-2, vedi sopra): non si può quindi escludere che una parte del rallentamento 2→3 worker derivi anche dalla minore memoria per nodo (16 GB → 8 GB, con conseguente maggiore spill su disco), ma il peggioramento 3→4 worker — stessa macchina, più nodi — isola in modo pulito l'effetto dello skew dall'effetto della memoria.

## 4.2 Effetto del partizionamento

Non essendo state eseguite le varianti di repartition (vedi nota in Sezione 3), non è possibile riportare un confronto quantitativo. Sulla base dell'analisi teorica in Sezione 4.1, ci si aspetterebbe che un maggior numero di partizioni non aiuti in modo sostanziale in questo caso: il problema non è un numero insufficiente di partizioni in generale, ma la concentrazione del carico su una singola chiave logica indipendentemente da quante partizioni totali esistono — un maggior `repartition()` ridistribuirebbe meglio le chiavi poco attive, ma non la chiave dominante, che resta comunque processata da un numero limitato di task.

## 4.3 Confronto tra i due approcci

A parità di worker (2, unico punto testato per entrambi con la stessa macchina), il self-join è risultato leggermente più lento del `groupByKey` (77.0 contro 74.2 minuti, +3.8%). Questo è coerente con quanto anticipato in Sezione 2.2: il self-join produce un prodotto cartesiano di tutte le località osservate nello stesso giorno prima di filtrare le coppie valide, generando quindi più lavoro intermedio del `groupByKey` (che invece genera le coppie già filtrate direttamente in memoria per ogni giorno). La differenza è comunque modesta a 2 worker; non è stato possibile verificare se si amplifichi a 3/4 worker per gli stessi limiti di tempo/quota descritti sopra. La correttezza del self-join è comunque stata verificata in ogni condizione testata (campione ridotto e cluster a 2 worker): risultato identico al `groupByKey` in tutti i casi.

## 5. Conclusioni

L'implementazione (in entrambe le varianti) individua correttamente la coppia di location arrotondate ((38.8, -122.8), (38.8, -122.7)) — un'area compatibile con il campo geotermico di The Geysers, in California del Nord — come la coppia con il maggior numero di co-occorrenze giornaliere (10032, dal 1990-01-05 al 2023-07-29). Il `groupByKey` si è dimostrato leggermente preferibile al `self-join` nell'unico confronto diretto disponibile (2 worker), coerentemente con il minor lavoro intermedio prodotto. Il risultato sperimentale più rilevante è tuttavia negativo rispetto alle aspettative di scalabilità naive: con una chiave fortemente skewata, aggiungere worker non solo non velocizza l'elaborazione oltre un certo punto, ma può peggiorarla per il maggiore overhead di rete, poiché né `groupByKey` né `self-join` su RDD offrono un meccanismo di redistribuzione delle chiavi calde. Un miglioramento naturale, lasciato come lavoro futuro, sarebbe la riformulazione del problema con `DataFrame/Dataset Spark SQL` per beneficiare dell'`Adaptive Query Execution` (skew join automatico), oppure l'applicazione manuale di una tecnica di key salting sulla chiave data per distribuire esplicitamente il lavoro della chiave calda su più task anche in RDD puro.

**Codice sorgente:** <https://github.com/Sh0wBlack32/SCP-earthquake-cooccurrence> (istruzioni per l'esecuzione su `DataProc` nel README del repository).